

Scanner 3D

Budiul Cristian-Carol

Ianuarie 2023

Cuprins

1. Specificația problemei	3
2. Soluția propusă	3
a. Proiectarea scannerului 3D	3
b. Colectarea resurselor necesare.....	4
c. Implementarea codului Arduino	4
d. Implementarea codului Processing	7
e. Proiectarea circuitului	9
f. Asamblare finală și concluzii.....	10
3. Bibliografie	11

1. Specificația problemei

De multe ori în timpul vieții, avem nevoie să măsurăm o cameră sau o bucată dintr-o cameră cu precizie și acuratețe, pentru a ne putea duce la bun sfârșit planurile. Dar, din păcate, de multe ori aceste dispozitive de măsurare și vizualizare a distanțelor măsurate sunt mult prea costisitoare pentru un proiect obișnuit. Din acest motiv, ne propunem să proiectăm un scanner 3D low-cost, care să poată fi folosit de oricine.

2. Soluția propusă

Din acest motiv, ne propunem să proiectăm un scanner 3D low-cost, care să poată fi folosit de oricine. Acest scanner va roti un senzor de distanță pe orizontală și pe verticală, trimițând în fiecare punct un mesaj prin intermediul interfeței seriale către computer. Mesajul va conține coordonatele măsurătorii și distanța măsurată de senzor. Pentru vizualizarea în timp real a distanțelor măsurate, vom interpreta aceste distanțe și vom construi o imagine colorată artificial în funcție de distanța măsurată.

Pentru a proiecta și implementa proiectul, am trecut prin o serie de pași de proiectare:

a. Proiectarea scannerului 3D

Am început proiectul prin proiectarea ansamblului 3D al scannerului. Din cauza experienței proprii în proiectarea sistemelor de acest gen, am folosit software-ul de modelare 3D parametric Fusion 360 pentru a proiecta ansamblul final.

Astfel, ansamblul final proiectat arată așa (Figura 1):

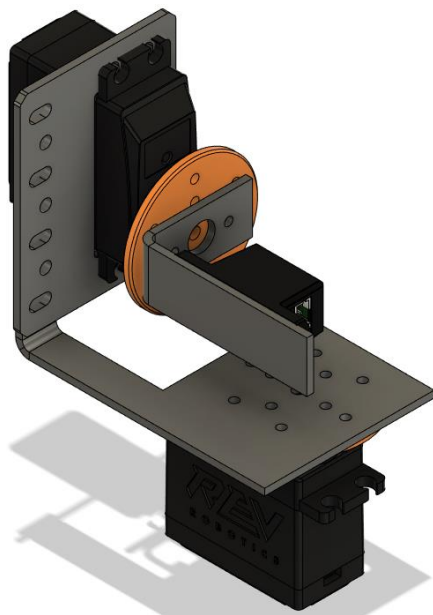


Figura 1. Modelul 3D al ansamblului

După cum se poate observa, vom folosi două servomotoare suprapuse pentru a obține rotațiile orizontale și verticale ale senzorului. Piese sunt conectate folosind piese de aluminiu îndoit, iar pentru conectarea tuturor pieselor vom folosi șuruburi și piulițe M3 și M4.

b. Colectarea resurselor necesare

Pentru a obține Arduino-ul, senzorul de distanță și servomotoarele necesare, am apelat la echipele de robotică FTC „RobotX Hunedoara” și „RavenTech”, din care am făcut parte în liceu. Acestea mi-au împrumutat următoarele piese:

- 2 servomotoare Digitale programabile goBILDA [1]
- 1 senzor de distanță cu laser de la Rev Robotics [2]
- 1 kit de Arduino, care conține 1 Arduino UNO R3, o cutie de baterii 4 * 1.5V, cablu USB și jumperi
- Feronerie

c. Implementarea codului Arduino

După obținerea resurselor necesare pentru testarea codului implementat, am realizat mai multe iterații de implementare și testare. Astfel, în urma iterațiilor am realizat următorul program Arduino care controlează servomotoarele și trimite valoarea citită de la senzor către calculator:

```
#include <Servo.h>
#include <VL53L0X.h>

VL53L0X sensor;

Servo yaw;
Servo pitch;

const int YAW_PIN = 9;
const int PITCH_PIN = 10;

int steps_yaw = 50;
int steps_pitch = 50;

int start_yaw = 30;
int start_pitch = 60;

int end_yaw = 150;
int end_pitch = 90;

int delay_yaw = 200;
int delay_pitch = 100;
```

```

int delay_initialisation = 1000;

int current_yaw;
int current_pitch;
int cstep_yaw;
int cstep_pitch;

bool yawDir = false;

void setup() {
    Serial.begin(9600);
    Wire.begin();
    yaw.attach(YAW_PIN);
    pitch.attach(PITCH_PIN);

    Serial.print("$steps ");
    Serial.print(steps_pitch);
    Serial.print(" ");
    Serial.println(steps_yaw);

    Serial.print("Initialising servos... ");
    pitch.write(start_pitch);
    yaw.write(start_yaw);
    delay(delay_initialisation);
    Serial.println("done!");

    Serial.print("Initialising sensor... ");
    sensor.setTimeout(500);
    if (!sensor.init()) {
        Serial.println("failed!");
        while (1) {}
    }
    sensor.startContinuous();
    Serial.println("done!");
}

void loop() {
    pitch_yaw_iterate();
}

void pitch_yaw_iterate() {
    for (cstep_pitch = 0; cstep_pitch <= steps_pitch; cstep_pitch++) {
        current_pitch = start_pitch + ((end_pitch - start_pitch) * cstep_pitch)
/ steps_pitch;
        pitch.write(current_pitch);
        delay(delay_pitch);
        yaw_iterate();
    }
}

```

```

    for (cstep_pitch = steps_pitch; cstep_pitch >= 0; cstep_pitch--) {
        current_pitch = start_pitch + ((end_pitch - start_pitch) * cstep_pitch)
/ steps_pitch;
        pitch.write(current_pitch);
        delay(delay_pitch);
        yaw_iterate();
    }
}

void yaw_iterate() {
    if (!yawDir) {
        for (cstep_yaw = 0; cstep_yaw <= steps_yaw; cstep_yaw++) {
            current_yaw = start_yaw + ((end_yaw - start_yaw) * cstep_yaw) /
steps_yaw;
            yaw.write(current_yaw);
            delay(delay_yaw);
            print_sensor_readout();
        }
    } else {
        for (cstep_yaw = steps_yaw; cstep_yaw >= 0; cstep_yaw--) {
            current_yaw = start_yaw + ((end_yaw - start_yaw) * cstep_yaw) /
steps_yaw;
            yaw.write(current_yaw);
            delay(delay_yaw);
            print_sensor_readout();
        }
    }
    yawDir = !yawDir;
}

void print_sensor_readout() {
    Serial.print("$distance ");
    Serial.print(cstep_pitch);
    Serial.print(" ");
    Serial.print(cstep_yaw);
    Serial.print(" ");
    Serial.println(sensor.readRangeContinuousMillimeters());
}

```

După cum se poate observa în program, la începutul acestuia avem multe valori de configurare cum ar fi numărul de pași pe care îi va face fiecare servomotor, unghiul inițial și final a acestora și durata dintre execuția fiecărui pas. De asemenea, am specificat ca întregi constanți și pinii pe care îi folosim pentru comunicarea cu servomotoarele

Pentru inițializare, am inițializat comunicarea serială (pentru calculator) și comunicarea I²C (pentru senzor). De asemenea, am trimis mesaje de inițializare care vor fi folosite pentru interfața grafică. După aceea, am făcut ceva setări pentru funcționarea senzorului.

Pentru bucla de rulare, am folosit funcții pentru a trimite datele la servomotor și pentru a mișca progresiv servomotoarele. Astfel, după fiecare mișcare a unui servomotor (pentru orizontală sau verticală) trimitem pasul curent de la fiecare servomotor și valoarea citită de la senzor (în mm).

Dar, pentru a afișa măsurătorile într-o imagine ușor de citit, este nevoie de un nou program...

d. Implementarea codului Processing

Pentru a implementa programul de afișare a valorilor în processing, am început prin a face o funcție de afișare a unei matrici de valori întregi pe ecran, asociind fiecărei valori numerice sub 1200 (care e distanța maximă în mm măsurată de senzor) o culoare între roșu (aproape) și verde (departe), folosind spațiul de culoare HSB.

După aceea, am implementat funcția setup, în care am inițializat un obiect nou de tip Serial, cu care să primim valorile trimise de Arduino. De asemenea, am făcut câteva setări folosite pentru buna funcționare a afișării, cum ar fi eliminarea marginilor dreptunghiurilor (noStroke()) sau colorarea background-ului în negru.

În funcția draw, desenăm încontinuu matricea de valori primite de la Arduino.

Pentru a primi în mod corect valorile de la Arduino, folosim funcția serialEvent. În cadrul acesteia, interpretăm comenzile primite de la Arduino (comenzi marcate cu \$), cum ar fi:

- \$steps <stepsX> <stepsY> - ne inițializează matricea de distanțe. Astfel, din cauza acestei inițializări, programul se adaptează la programul de Arduino, orice modificare adusă numărului de pași în programul Arduino fiind injectată și în cadrul acestui program.
- \$distance <row> <col> <value> - inserează valoarea „value” în matrice, la rândul „row” și coloana „col”.

Codul final implementat în Processing este următorul:

```
import processing.serial.*;

Serial arduino;

int w = 500;
int h = 500;

int rows;
int cols;
int[][] distanceMatrix;

void setup(){
    size(500, 500);
    String portName = Serial.list()[Serial.list().length-1];
    arduino = new Serial(this, portName, 9600);
    arduino.bufferUntil('\n');
    colorMode(HSB, 360, 100, 100);
    noStroke();
    background(0);
}

void draw(){
    displayMatrix(distanceMatrix);
}

void serialEvent(Serial ardu){
    String inputString = ardu.readStringUntil('\n');
    inputString = inputString.trim();
    println(inputString);

    String[] splitted = split(inputString, ' ');
    if(splitted[0].charAt(0) == '$'){
        splitted[0] = splitted[0].substring(1);
        if(splitted[0].equals("steps")){
            rows = int(splitted[1]) + 1;
            cols = int(splitted[2]) + 1;
            distanceMatrix = new int[rows][cols];
            println(rows + " " + cols);
        } else if(splitted[0].equals("distance") && distanceMatrix !=
null){
            distanceMatrix[int(splitted[1])][int(splitted[2])] =
int(splitted[3]);
            printMatrix(distanceMatrix);
        }
    }
}

void printMatrix(int[][] m){
    for(int i = 0; i < m.length; i++){
        for(int j = 0; j < m[i].length; j++){
            print(m[i][j] + " ");
        }
        println();
    }
}

void displayMatrix(int[][] m){
    if(m != null){
        for(int i = 0; i < rows; i++){
```



```

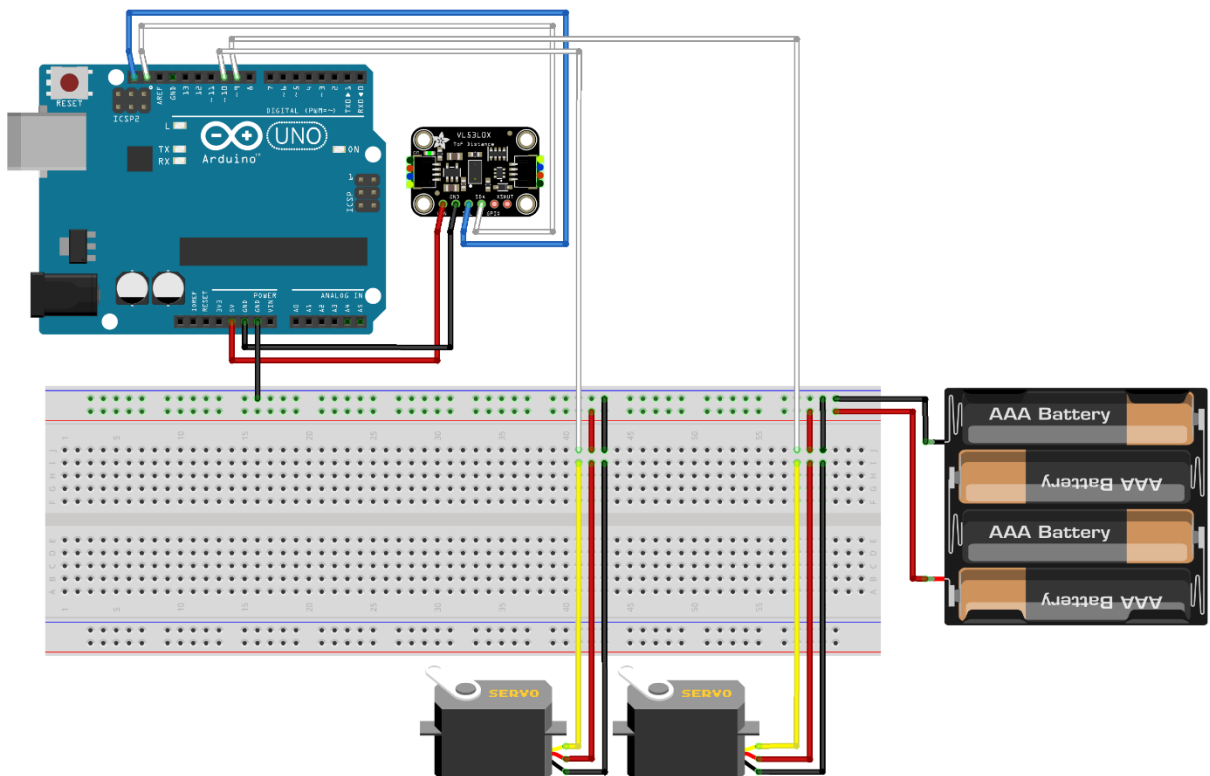
for(int j = 0; j < cols; j++){
    int cellX = w * j / cols;
    int cellWidth = w / cols + 1;
    int cellY = h * i / rows;
    int cellHeight = h / rows + 1;
    int hue = m[i][j] > 1200 ? 120 : ((120 * m[i][j]) / 1200);
    if(m[i][j] != 0){
        fill(color(hue, 100, 100));
        rect(cellX, cellY, cellWidth, cellHeight);
    }
}
}
}
}

```

e. Proiectarea circuitului

Pentru proiectarea circuitului, am folosit Fritzing și documentația online a senzorului și a Arduino-ului. Aceasta a fost una dintre cele mai ușoare părți ale proiectului, fiind nevoie doar să legăm toate componentele la Arduino. Singurele părți mai speciale ale circuitului sunt reprezentate de cutia de baterii (care reprezintă alimentarea pentru servomotoare) și conexiunea dintre pinul GND al Arduino și polul minus al cutiei cu baterii (aceasta asigurând un ground comun, pentru comunicare corectă a semnalelor PWM către servomotoare)

Circuitul final este reprezentat de această schemă (Figura 2):



fritzing

Figura 2. Schema circuitului

f. Asamblare finală și concluzii

Proiectul asamblat în totalitate și funcțional arată așa (Figura 3):

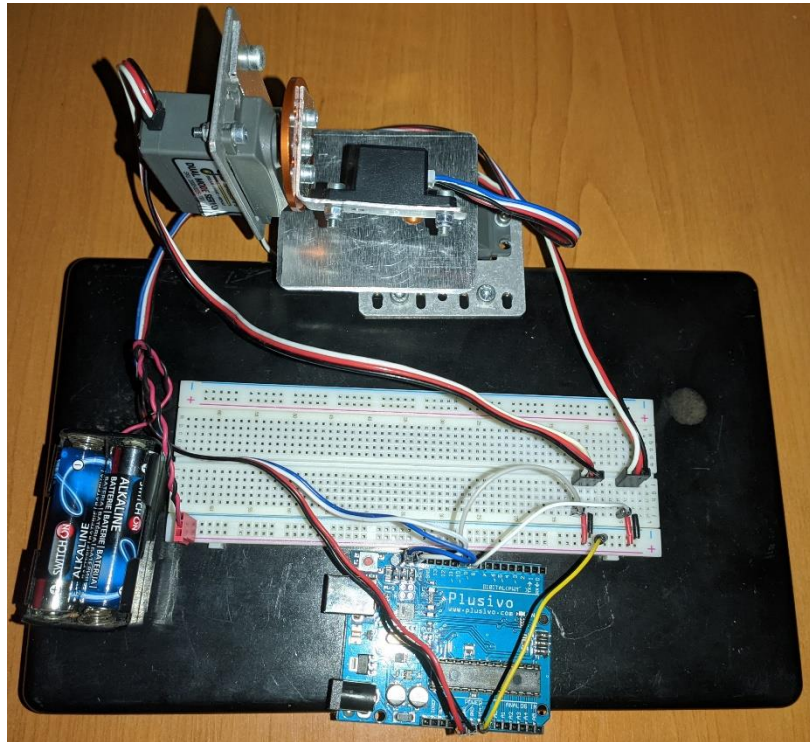


Figura 3. Proiectul asamblat

Primele imagini (de rezoluție mai mare) realizate de proiect au fost acestea (Figura 4):

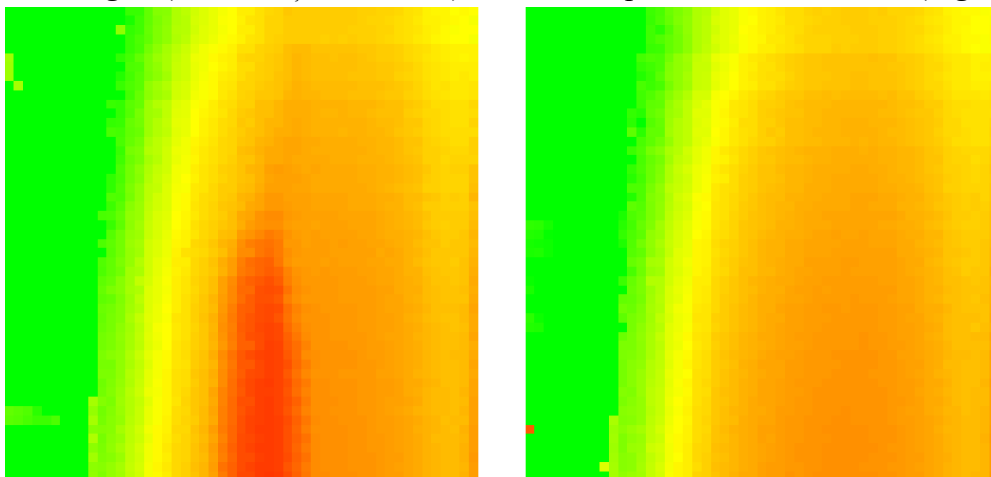


Figura 4. Măsurători efectuate de proiect
stânga, un perete cu o sticlă de apă în fața lui; dreapta, același perete dar fără sticla de apă

În concluzie, am reușit să realizăm un scanner 3D low-cost, care conține părți ușor accesibile și care are o precizie acceptabilă. Ca dezvoltări ulterioare, proiectul ar putea fi miniaturizat și/sau îmbunătățit în mai multe moduri, cum ar fi:

- Folosirea de motoare stepper pentru controlul mult mai fin al orientării
- Folosirea unui senzor de distanță mai precis (atât unghiular cât și ca distanță) și/sau cu o plajă de măsurători mai largă

3. Bibliografie

- [1] goBILDA, „2000 Series Dual Mode Servo (25-2, Torque),” January 2023. [Interactiv].
Available: <https://www.gobilda.com/2000-series-dual-mode-servo-25-2-torque/>.
- [2] R. Robotics, „2m Distance Sensor,” REV Robotics, January 2023. [Interactiv].
Available: <https://www.revrobotics.com/rev-31-1505/>.